

LAPORAN PRATIKUM
STRUKTUR DATA
“SORTING 1 (Insertion, Selection, Bubble)”
PEKAN KE-7
disusun Oleh:
Aisyah Najmiatul Fauziah
NIM 2511533030
Dosen Pengampu: DR. WAHYUDI, S.T, M.T
Asisten Pratikum: Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur dipanjatkan ke hadirat Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya, sehingga laporan praktikum Struktur Data dengan topik “Sorting 1 (Insertion, Selection, Bubble)” ini dapat diselesaikan dengan baik dan tepat waktu.

Laporan praktikum ini disusun sebagai salah satu syarat akademis serta bukti tertulis atas pelaksanaan praktikum Struktur Data pada pekan ke-7. Melalui penyusunan laporan ini, diharapkan pemahaman mengenai konsep dasar algoritma pengurutan data beserta implementasinya dalam pemrograman dapat terdokumentasi dengan sistematis.

Diucapkan terima kasih yang sebesar-besarnya kepada dosen pengampu mata kuliah Struktur Data serta para asisten laboratorium yang telah memberikan bimbingan, arahan, dan ilmu yang bermanfaat selama proses perkuliahan dan praktikum berlangsung. Draf laporan ini disadari masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang.

Akhir kata, semoga laporan praktikum ini dapat memberikan manfaat nyata, menambah wawasan, dan menjadi referensi yang berguna bagi pembaca, khususnya dalam memahami algoritma pengurutan data.

Padang, 20 Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I	1
PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	1
BAB II.....	2
PEMBAHASAN	2
2.1 Praktikum “BubleSort_2511533030”.....	2
2.2 Praktikum “InsertionSort_2511533030”	2
2.3 Praktikum “SelectionSort_2511533030”	2
2.4 Pratikum “InsertionSortGUI_2511533030”	3
BAB III.....	3
KESIMPULAN.....	3
3.1 Kesimpulan.....	3

BAB I

PENDAHULUAN

1.1 Latar Belakang

Di dalam dunia ilmu komputer dan pengelolaan basis data, penyimpanan informasi secara acak sering kali menurunkan efisiensi sistem saat melakukan pencarian atau pemrosesan data. Proses pengurutan data (*sorting*) menjadi salah satu operasi paling mendasar dan krusial yang digunakan untuk menyusun kembali sekumpulan data acak ke dalam urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*).

Pengurutan yang efisien memegang peran penting dalam mengoptimalkan performa algoritma lain, seperti algoritma pencarian biner (*binary search*) yang mengharuskan data berada dalam kondisi terurut. Terdapat berbagai macam metode pengurutan konvensional yang menjadi pondasi dasar pemrograman, di antaranya adalah *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.

- **Bubble Sort** bekerja dengan cara membandingkan elemen yang berdampingan secara berulang dan menukarnya jika posisinya salah, layaknya gelembung yang naik ke permukaan.
- **Selection Sort** berfokus pada pencarian elemen ekstrem (terkecil atau terbesar) dari bagian yang belum terurut untuk kemudian diletakkan pada posisi yang tepat.
- **Insertion Sort** menyisipkan elemen satu per satu ke dalam bagian data yang sudah terurut, mirip dengan cara seseorang merapikan kartu di tangan.

Pemahaman mendalam mengenai karakteristik, alur logika, tingkat kompleksitas waktu (*time complexity*), serta efisiensi penggunaan memori dari ketiga algoritma ini sangat penting bagi seorang mahasiswa informatika atau sistem informasi. Oleh karena itu, praktikum pekan ke-7 ini dilaksanakan untuk menjembatani teori yang didapat di kelas dengan implementasi kode pemrograman secara riil di dalam laboratorium.

1.2 Tujuan Praktikum

1. Memahami konsep dasar, logika berpikir, dan mekanisme kerja dari algoritma *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*.
2. Mampu mengimplementasikan algoritma *Insertion*, *Selection*, dan *Bubble Sort* ke dalam bahasa pemrograman yang digunakan saat praktikum.
3. Menganalisis perbedaan performa dan tahapan (*pass*) yang terjadi pada masing-masing metode pengurutan dalam menyelesaikan suatu kasus data acak.

1.3 Manfaat Praktikum

1. Meningkatkan keterampilan teknis (*hard skill*) dalam menulis kode program pengurutan data yang valid dan bebas dari *error*.
2. Membantu mahasiswa memahami kapan waktu yang tepat untuk memilih salah satu dari ketiga algoritma tersebut berdasarkan kondisi data mentah yang dihadapi.

3. Mengasah kemampuan analisis algoritma, khususnya dalam melihat jumlah iterasi dan pergeseran elemen data.

BAB II

PEMBAHASAN

2.1 Praktikum “BubleSort_2511533030”

Langkah Kerja :

1. Buat Package dan Class baru
2. Buat kode berikut

```
package pekan7_2511533030;

public class BubleSort_2511533030 {
    public static void bubbleSort(int[] arr_3030) {
        int n_3030 = arr_3030.length;
        for (int i_3030 = 0; i_3030 < n_3030; i_3030++) {
            for (int j_3030 = 0; j_3030 < n_3030 - i_3030 - 1; j_3030++) {
                if (arr_3030[j_3030] > arr_3030[j_3030 + 1]) {
                    int temp_3030 = arr_3030[j_3030];
                    arr_3030[j_3030] = arr_3030[j_3030 + 1];
                    arr_3030[j_3030 + 1] = temp_3030;
                    // System.out.println("data:"+arr[j]+" "+arr[j+1]);
                }
            }
        }
    }

    public static void main(String[] args) {
        int arr_3030[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_3030 = arr_3030.length;
        System.out.print("array yang belum terurut: ");
        for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
            System.out.print(arr_3030[i_3030] + " ");
        System.out.println("");
        bubbleSort(arr_3030);
        System.out.print("array yang terurut menggunakan BubbleSort: ");
        for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
            System.out.print(arr_3030[i_3030] + " ");
        System.out.println("");
    }
}
```

Gambar 2.1 kode program “BubleSort_2511533030”

3. Didapatkan output sebagai berikut

```
<terminated> BubleSort_2511533030 [Java Application] C:\Users\AISYAH NAJMIATUL F\p2\pool\plugins\org
array yang belum terurut: 23 78 45 8 32 56 1
array yang terurut menggunakan BubbleSort: 1 8 23 32 45 56 78
```

Gambar 2.1 output program “BubleSort_2511533030”

Penjelasan Kode :

2.1.1 Deklarasi Package dan Class

- berfungsi sebagai penentu struktur identitas file.

```
package pekan7_2511533030;
public class BubleSort_2511533030 {
```

2.1.2 Pembentukan Gelembung (Fungsi BubleSort)

- mengukur panjang deretan angka acak dan menyiapkan **perulangan bertingkat (nested loop)**.

- Perulangan luar mengatur seberapa banyak putaran pemindaian penuh yang harus dilakukan.
- Perulangan dalam bertugas menyisir elemen dari indeks awal hingga ke batas data yang belum terurut

```
public static void bubbleSort(int[] arr_3030) {
    int n_3030 = arr_3030.length;
    for (int i_3030 = 0; i_3030 < n_3030; i_3030++) {
        for (int j_3030 = 0; j_3030 < n_3030 - i_3030 - 1; j_3030++) {
```

2.1.3 Perbandingan dan Penukaran Posisi (Swapping)

- Perintah **if** akan membandingkan dua angka yang letaknya bersebelahan.
- Jika angka di sebelah kiri ternyata lebih besar daripada angka di sebelah kanannya, posisi keduanya akan **ditukar (swap)** menggunakan variabel penampung sementara (**temp_3030**).
- roses ini membuat angka besar perlahan bergeser ke kanan

```
if (arr_3030[j_3030] > arr_3030[j_3030 + 1]) {
    int temp_3030 = arr_3030[j_3030];
    arr_3030[j_3030] = arr_3030[j_3030 + 1];
    arr_3030[j_3030 + 1] = temp_3030;
    // System.out.println("data:"+arr[j]+" "+arr[j+1]);
}
```

2.1.4 Inisialisasi Data dan Penampilan Awal (Awal Fungsi main)

- Di sini, kita menyiapkan susunan angka-angka acak (23, 78, 45, ...) di dalam memori dan langsung mencetaknya ke layar komputer menggunakan perulangan **for** agar pengguna bisa melihat kondisi susunan data sebelum diurutkan.

```
public static void main(String[] args) {
    int arr_3030[] = { 23, 78, 45, 8, 32, 56, 1 };
    int n_3030 = arr_3030.length;
    System.out.print("array yang belum terurut: ");
    for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
        System.out.print(arr_3030[i_3030] + " ");
    System.out.println("");
}
```

2.1.5 Akhir Fungsi Main

- Bertugas memanggil fungsi bubbleSort(arr_3030) untuk mengolah data acak tadi hingga berurutan rapi.
- etelah proses pengurutan di latar belakang selesai, deretan angka yang kini sudah rapi dari terkecil ke terbesar (1, 8, 23, ...) dicetak kembali ke layar sebagai hasil final praktikum.

```
bubbleSort(arr_3030);
System.out.print("array yang terurut menggunakan BubbleSort: ");
for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
    System.out.print(arr_3030[i_3030] + " ");
System.out.println("");
}
```

2.2 Praktikum “InsertionSort_2511533030”

Langkah Kerja :

1. Buat Class Baru
2. Buat Kode berikut

```

package pekan7_2511533030;

public class InsertionSort_2511533030 {
    public static void insertionSort(int[] arr_3030) {
        int n_3030 = arr_3030.length;
        for (int i_3030 = 1; i_3030 < n_3030; i_3030++) {
            int key_3030 = arr_3030[i_3030];
            int j_3030 = i_3030 - 1;
            while (j_3030 >= 0 && arr_3030[j_3030] > key_3030) {
                arr_3030[j_3030 + 1] = arr_3030[j_3030];
                j_3030--;
            }
            arr_3030[j_3030 + 1] = key_3030;
        }
    }

    public static void main(String[] args) {
        int arr_3030 [] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_3030 = arr_3030.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
            System.out.print(arr_3030[i_3030] + " ");
        System.out.println("");
        insertionSort(arr_3030);
        System.out.printf("array yang terurut:\n");
        for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
            System.out.print(arr_3030[i_3030] + " ");
        System.out.println("");
    }
}

```

Gambar 2.2 kode program “InsertionSort_2511533030”

3. Didapatkan output sebagai berikut

```

<terminated> InsertionSort_2511533030 [Java Application] C:\Users\AISYAH NAJM
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

Gambar 2.2 output program “InsertionSort_2511533030”

Penjelasan Kode :

2.2.1 Deklarasi Package dan Class

- mengatur penamaan struktur file
- mendeklarasikan nama modul utama program

```

package pekan7_2511533030;
public class InsertionSort_2511533030 {

```

2.2.2 Penentuan Target Sisip (Awal Fungsi insertionSort)

- memulai proses pengurutan kartu/data
- Perulangan **for** dimulai dari indeks ke-1 (bukan 0) karena elemen pertama dianggap sudah berada di posisi yang benar untuk sementara.

- Variabel `key_3030` berfungsi memegang angka target yang sedang diuji, dan `j_3030` disiapkan sebagai indeks penunjuk angka-angka di sebelah kirinya yang bertindak sebagai area data yang sudah terurut.

```
public static void insertionSort(int[] arr_3030) {
    int n_3030 = arr_3030.length;
    for (int i_3030 = 1; i_3030 < n_3030; i_3030++) {
        int key_3030 = arr_3030[i_3030];
        int j_3030 = i_3030 - 1;
```

2.2.3 Logika Pergeseran Data dan Penyisipan (While Loop)

- Kelompok ini melakukan penyesiran ke arah belakang (kiri).
- Selama angka di sebelah kiri (`arr_3030[j_3030]`) nilainya lebih besar daripada angka target (`key_3030`), maka angka yang lebih besar tersebut akan **digeser ke kanan** (`arr_3030[j_3030 + 1] = arr_3030[j_3030]`).
- Setelah ruang kosong yang pas ditemukan (kondisi *while* terhenti), angka target **key_3030** langsung **disisipkan** ke posisi akhir yang tepat.

```
while (j_3030 >= 0 && arr_3030[j_3030] > key_3030) {
    arr_3030[j_3030 + 1] = arr_3030[j_3030];
    j_3030--;
}
arr_3030[j_3030 + 1] = key_3030;
}
```

2.2.4 Pembuatan Data Mentah dan Cetak Awal (Awal Fungsi main)

- memanfaatkan perulangan `for` untuk mencetak deretan angka mentah tersebut ke layar konsol agar kita bisa melihat bentuk data sebelum algoritma bekerja.

```
public static void main(String[] args) {
    int arr_3030 [] = { 23, 78, 45, 8, 32, 56, 1 };
    int n_3030 = arr_3030.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
        System.out.print(arr_3030[i_3030] + " ");
    System.out.println("");
}
```

2.2.5 Pemanggilan Fungsi Pemroses dan Cetak Hasil Akhir (Akhir Fungsi main)

- penutup ini bertugas memanggil prosedur `insertionSort(arr_3030)` untuk mengurutkan angka acak tadi di latar belakang.
- Begitu proses penyisipan selesai dan data telah tersusun rapi dari yang terkecil ke terbesar, perulangan terakhir akan mencetak hasil akhir

```
insertionSort(arr_3030);
System.out.printf("array yang terurut:\n");
for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
    System.out.print(arr_3030[i_3030] + " ");
System.out.println("");
}
```

2.3 Praktikum “SelectionSort_2511533030”

Langkah Kerja :

1. Buat Class Baru
2. Buat Kode berikut

```
package pekan7_2511533030;
```

```

public class SelectionSort_2511533030 {
    public static void selectionSort(int[] arr_3030) {
        int n_3030 = arr_3030.length;
        for (int i_3030 = 0; i_3030 < n_3030; i_3030++) {
            int minIndex = i_3030;
            for (int j_3030 = i_3030 + 1; j_3030 < n_3030; j_3030++) {
                if (arr_3030[j_3030] < arr_3030[minIndex]) {
                    minIndex = j_3030;
                }
            }
            int temp_3030 = arr_3030[i_3030];
            arr_3030[i_3030] = arr_3030[minIndex];
            arr_3030[minIndex] = temp_3030;
        }
    }
    public static void main(String[] args) {
        int arr_3030[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_3030 = arr_3030.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
            System.out.print(arr_3030[i_3030] + " ");
        System.out.println("");
        selectionSort(arr_3030);
        System.out.printf("array yang terurut:\n");
        for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
            System.out.print(arr_3030[i_3030] + " ");
        System.out.println("");
    }
}

```

Gambar 2.3 kode program “SelectionSort_2511533030”

3. Didapatkan Output Sebagai Berikut

```

<terminated> SelectionSort_2511533030 [Ja
array yang belum terurut:
23 78 45 8 32 56 1
array yang terurut:
1 8 23 32 45 56 78

```

Gambar 2.3 output program “SelectionSort_2511533030”

Penjelasan Kode :

2.3.1 Deklarasi Package dan Class

- berfungsi sebagai deklarasi alamat file
- Struktur public class digunakan untuk mendefinisikan nama kelas utama

```

package pekan7_2511533030;
public class SelectionSort_2511533030 {

```

2.3.2 Pencarian Nilai Minimum Terkecil (Awal Fungsi selectionSort)

- melakukan "seleksi" atau pencarian angka paling kecil dari sisa data yang belum terurut.
- Perulangan luar (**i_3030**) menentukan batas awal area data yang sedang diperiksa, dan mengasumsikan indeks tersebut sebagai pemilik angka terkecil (**minIndex**).

- Perulangan dalam (**j_3030**) akan menyisir seluruh data ke kanan; jika ditemukan angka yang jauh lebih kecil.

```
public static void selectionSort(int[] arr_3030) {
    int n_3030 = arr_3030.length;
    for (int i_3030 = 0; i_3030 < n_3030; i_3030++) {
        int minIndex = i_3030;
        for (int j_3030 = i_3030 + 1; j_3030 < n_3030; j_3030++) {
            if (arr_3030[j_3030] < arr_3030[minIndex]) {
                minIndex = j_3030;
            }
        }
    }
}
```

2.3.3 Penukaran Posisi Elemen Final (Swapping)

- Setelah perulangan terdalam selesai memastikan posisi angka terkecil yang sebenarnya di putaran tersebut
- kelompok ini mengeksekusi penukaran tempat (*swap*).
- Angka terkecil yang ditemukan di minIndex ditukar posisinya ke batas depan data yang belum urut (**i_3030**) menggunakan bantuan wadah penampung sementara (**temp_3030**).
- sisi kiri array secara bertahap terisi oleh angka-angka yang sudah pasti terurut.

```
int temp_3030 = arr_3030[i_3030];
arr_3030[i_3030] = arr_3030[minIndex];
arr_3030[minIndex] = temp_3030;
}
```

2.3.4 Alokasi Data Mentah dan Penampilan Awal (Awal Fungsi main)

- Merupakan titik awal jalannya program utama]
- menginisialisasi atau memesan sebuah larik (*array*) berisi 7 angka acak ke dalam memori.
- Selanjutnya, program melakukan perulangan for singkat untuk mencetak kondisi deretan angka mentah tersebut ke layar monitor agar dapat dibandingkan dengan hasil akhir nanti

```
}
public static void main(String[] args) {
    int arr_3030[] = { 23, 78, 45, 8, 32, 56, 1 };
    int n_3030 = arr_3030.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
        System.out.print(arr_3030[i_3030] + " ");
    System.out.println("");
}
```

2.3.5 Pemrosesan Utama dan Cetak Hasil Akhir (Akhir Fungsi main)

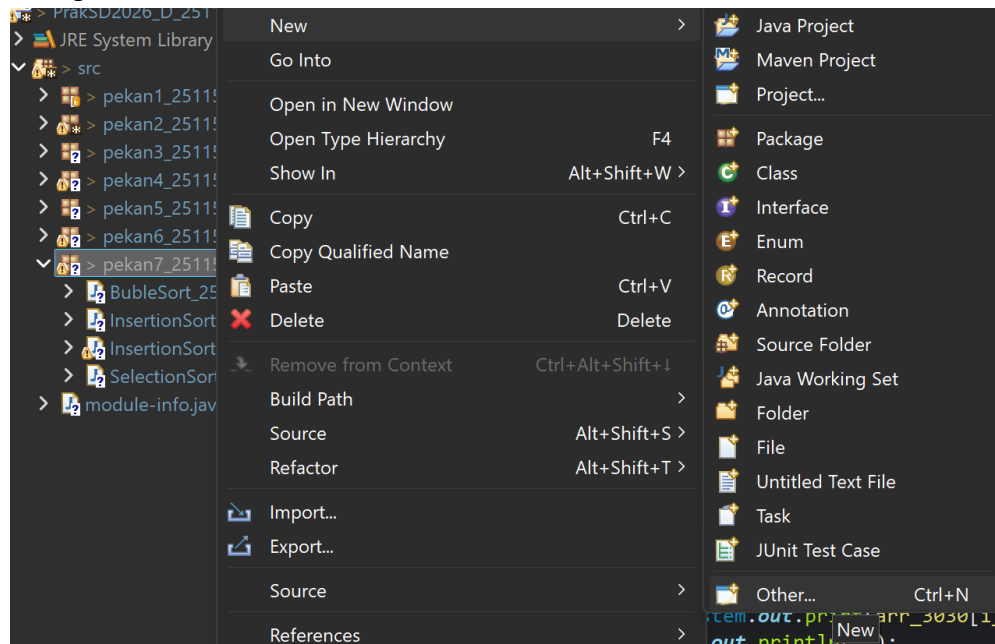
- Kelompok penutup ini mengarahkan data acak ke dalam prosedur selectionSort(**arr_3030**) agar diproses menggunakan metode seleksi.
- Begitu komputasi selesai dan seluruh elemen di memori sudah tersusun secara menaik (*ascending*), deretan angka yang telah rapi (1 8 23 32 45 56 78) dicetak ke baris output paling akhir.

```
selectionSort(arr_3030);
System.out.printf("array yang terurut:\n");
for (int i_3030 = 0; i_3030 < n_3030; i_3030++)
    System.out.print(arr_3030[i_3030] + " ");
System.out.println("");
}
```

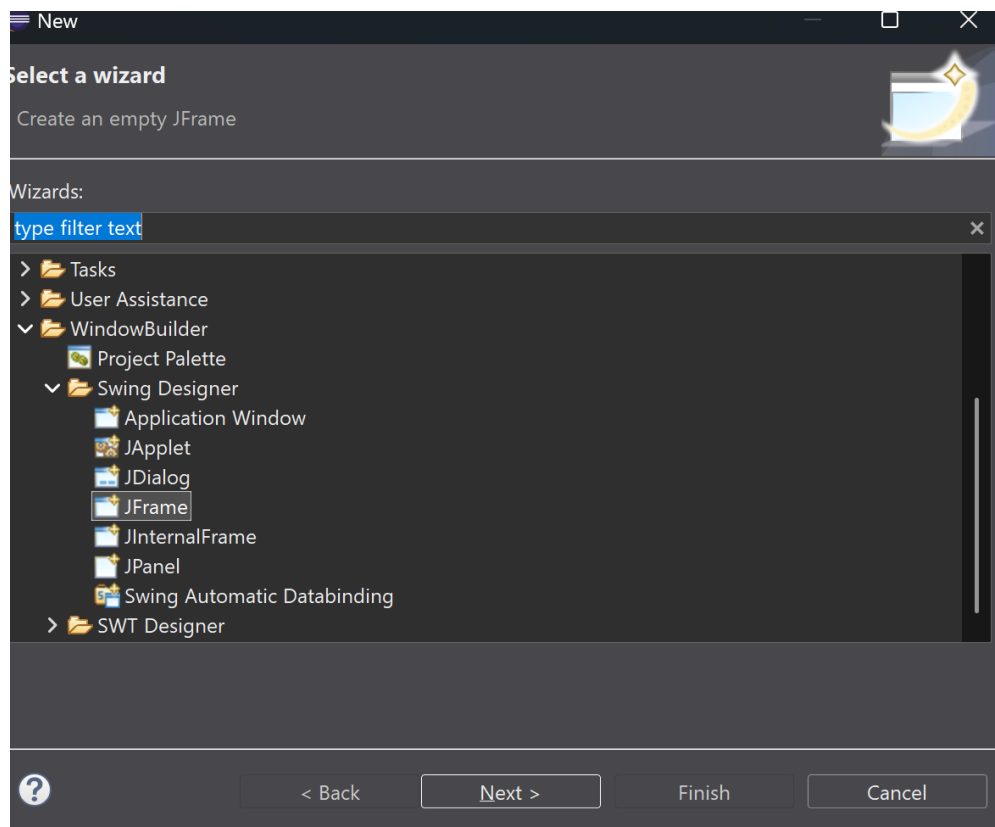
2.3 Pratikum “InsertionSortGUI_2511533030”

Langkah Kerja :

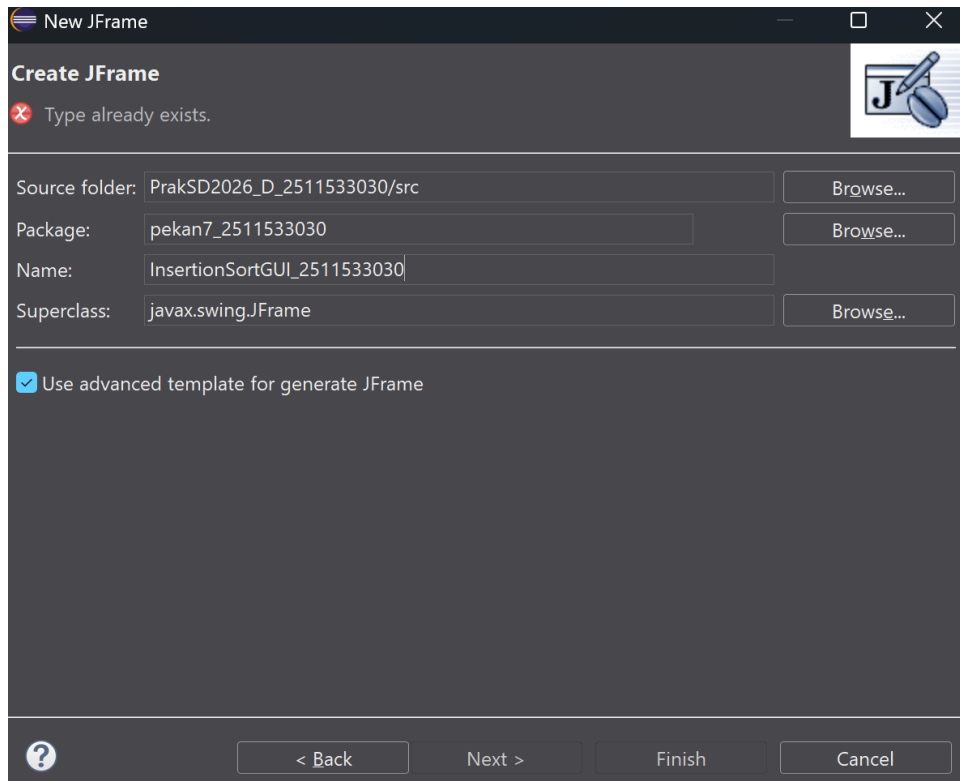
1. Klik pekan 7, lalu klik ‘New’ dan klik other



2. Lalu klik ‘JFrame’



3. Buat Class Baru



4. Lalu akan muncul tampilan berikut, dan hapus pada bagian Tengah nya

```

1 package pekan7_2511533030;
2
3 import java.awt.EventQueue;
4
5 public class InsertionSortGUI extends JFrame {
6
7     private static final long serialVersionUID = 1L;
8     private JPanel contentPane;
9
10    /**
11     * Launch the application.
12     */
13    public static void main(String[] args) {
14        EventQueue.invokeLater(new Runnable() {
15            public void run() {
16                try {
17                    InsertionSortGUI frame = new InsertionSortGUI();
18                    frame.setVisible(true);
19                } catch (Exception e) {
20                    e.printStackTrace();
21                }
22            }
23        });
24    }
25
26    /**
27     * Create the frame.
28     */
29    public InsertionSortGUI() {
30        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
31        setBounds(100, 100, 450, 300);
32        contentPane = new JPanel();
33        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
34        setContentPane(contentPane);
35    }
36
37 }

```

5. Lalu buat kode berikut

```

public InsertionSortGUI_2511533030() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane_3030 = new JPanel();
    contentPane_3030.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane_3030);
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
}

```

```

        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_3030 = new JPanel(new FlowLayout());
        inputField_3030 = new JTextField(30);
        setButton_3030 = new JButton("Set Array");
        inputPanel_3030.add(new JLabel("Masukkan angka (pisahkan dengan
koma):"));

        inputPanel_3030.add(inputField_3030);
        inputPanel_3030.add(setButton_3030);

        // Panel array visual
        panelArray_3030 = new JPanel();
        panelArray_3030.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_3030 = new JPanel();
        stepButton_3030 = new JButton("Langkah Selanjutnya");
        resetButton_3030 = new JButton("Reset");
        stepButton_3030.setEnabled(false);
        controlPanel_3030.add(stepButton_3030);
        controlPanel_3030.add(resetButton_3030);

        // Area teks untuk log langkah-langkah
        stepArea_3030 = new JTextArea(8, 60);
        stepArea_3030.setEditable(false);
        stepArea_3030.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPanel_3030 = new JScrollPane(stepArea_3030);

        // Tambahkan panel ke frame
        add(inputPanel_3030, BorderLayout.NORTH);
        add(panelArray_3030, BorderLayout.CENTER);
        add(controlPanel_3030, BorderLayout.SOUTH);
        add(scrollPanel_3030, BorderLayout.EAST);

        // Event Set Array
        setButton_3030.addActionListener(e -> setArrayFromInput());

        // Event Langkah Selanjutnya
        stepButton_3030.addActionListener(e -> performStep());

        // Event Reset
        resetButton_3030.addActionListener(e -> reset());

    }

    private void setArrayFromInput() {
        String text = inputField_3030.getText().trim();
        if (text.isEmpty()) return;

        String[] parts = text.split(",");
        array_3030 = new int[parts.length];

        try {
            for (int k = 0; k < parts.length; k++) {
                array_3030[k] = Integer.parseInt(parts[k].trim());
            }
        } catch (NumberFormatException e) {

```

```

        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan "
        + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    i_3030 = 1;
    sorting_3030 = true;
    stepButton_3030.setEnabled(true);
    stepArea_3030.setText("");

    panelArray_3030.removeAll();
    labelArray_3030 = new JLabel[array_3030.length];

    for (int k_3030 = 0; k_3030 < array_3030.length; k_3030++) {
        labelArray_3030[k_3030] = new
JLabel(String.valueOf(array_3030[k_3030]));
        labelArray_3030[k_3030].setFont(new Font("Arial", Font.BOLD, 24));

labelArray_3030[k_3030].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_3030[k_3030].setPreferredSize(new Dimension(50, 50));

labelArray_3030[k_3030].setHorizontalAlignment(SwingConstants.CENTER);

        panelArray_3030.add(labelArray_3030[k_3030]);
    }

    panelArray_3030.revalidate();
    panelArray_3030.repaint();
}
private void performStep() {
    if (i_3030 < array_3030.length && sorting_3030) {
        int key_3030 = array_3030[i_3030];
        j_3030 = i_3030 - 1;

        StringBuilder stepLog = new StringBuilder();
        Object stepCount;
        stepLog.append("Langkah ").append(stepCount_3030).
            append(": Memasukkan ").append(key_3030).append("\n");

        while (j_3030 >= 0 && array_3030[j_3030] > key_3030) {
            array_3030[j_3030 + 1] = array_3030[j_3030];
            j_3030--;
        }

        array_3030[j_3030 + 1] = key_3030;

        updateLabels();
        stepLog.append("Hasil:
").append(arrayToString(array_3030)).append("\n\n");
        stepArea_3030.append(stepLog.toString());

        i_3030++;
        stepCount_3030++;

        if (i_3030 == array_3030.length) {
            sorting_3030 = false;
            stepButton_3030.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
        }
    }
}

```

```

    }
}
}
private void updateLabels() {
    for (int k_3030 = 0; k_3030 < array_3030.length; k_3030++) {
labelArray_3030[k_3030].setText(String.valueOf(array_3030[k_3030]));
    }
}

private void reset() {
    inputField_3030.setText("");
    panelArray_3030.removeAll();
    panelArray_3030.revalidate();
    panelArray_3030.repaint();
    stepArea_3030.setText("");
    stepButton_3030.setEnabled(false);
    sorting_3030 = false;
    i_3030 = 1;
    stepCount_3030 = 1;
}

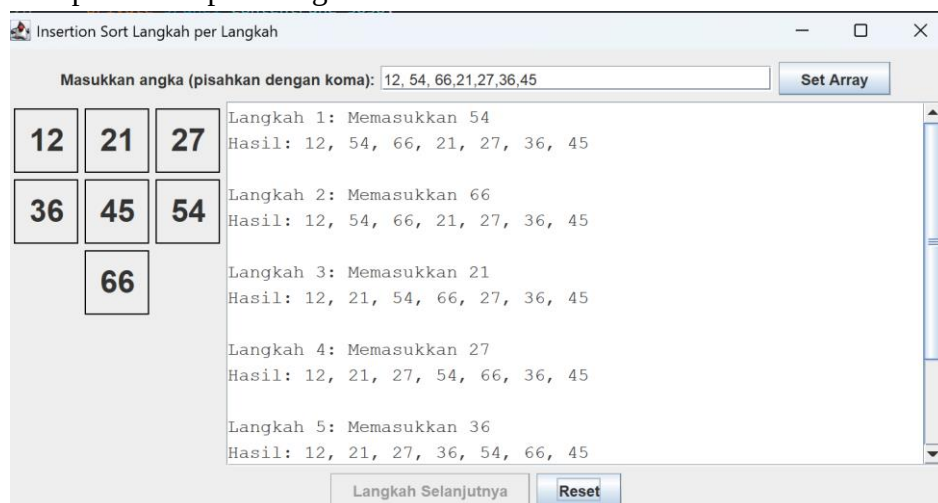
private String arrayToString(int[] arr) {
    StringBuilder sb = new StringBuilder();
    for (int k_3030 = 0; k_3030 < arr.length; k_3030++) {
        sb.append(arr[k_3030]);
        if (k_3030 < arr.length - 1) sb.append(", ");
    }
    return sb.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511533030 gui_3030 = new
InsertionSortGUI_2511533030();
        gui_3030.setVisible(true);
    });
}
}

```

Gambar 2.4 kode program “InsertionSortGUI_2511533030”

6. Didapatkan output sebagai berikut



Gambar 2.4 output program “InsertionSortGUI_2511533030”

Penjelasan Program :

2.4.1 Konstruktor dan Pengaturan Tampilan Utama (InsertionSortGUI_2511533030)

- pondasi utama pembangunan jendela aplikasi (*User Interface*)
- Di sini kamu mengatur ukuran bingkai jendela, tata letak posisi komponen (***BorderLayout***), membuat kolom teks input, tombol-tombol interaktif, serta area log teks di sisi kanan.

```
public InsertionSortGUI_2511533030() {
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setBounds(100, 100, 450, 300);
    contentPane_3030 = new JPanel();
    contentPane_3030.setBorder(new EmptyBorder(5, 5, 5, 5));
    setContentPane(contentPane_3030);
    setTitle("Insertion Sort Langkah per Langkah");
    setSize(750, 400);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setLocationRelativeTo(null);
    setLayout(new BorderLayout());

    // Panel input
    JPanel inputPanel_3030 = new JPanel(new FlowLayout());
    inputField_3030 = new JTextField(30);
    setButton_3030 = new JButton("Set Array");
    inputPanel_3030.add(new JLabel("Masukkan angka (pisahkan dengan
koma):"));

    inputPanel_3030.add(inputField_3030);
    inputPanel_3030.add(setButton_3030);

    // Panel array visual
    panelArray_3030 = new JPanel();
    panelArray_3030.setLayout(new FlowLayout());

    // Panel kontrol
    JPanel controlPanel_3030 = new JPanel();
    stepButton_3030 = new JButton("Langkah Selanjutnya");
    resetButton_3030 = new JButton("Reset");
    stepButton_3030.setEnabled(false);
    controlPanel_3030.add(stepButton_3030);
    controlPanel_3030.add(resetButton_3030);

    // Area teks untuk log langkah-langkah
    stepArea_3030 = new JTextArea(8, 60);
    stepArea_3030.setEditable(false);
    stepArea_3030.setFont(new Font("Monospaced", Font.PLAIN, 14));
    JScrollPane scrollPanel_3030 = new JScrollPane(stepArea_3030);

    // Tambahkan panel ke frame
    add(inputPanel_3030, BorderLayout.NORTH);
    add(panelArray_3030, BorderLayout.CENTER);
    add(controlPanel_3030, BorderLayout.SOUTH);
    add(scrollPanel_3030, BorderLayout.EAST);

    // Event Set Array
    setButton_3030.addActionListener(e -> setArrayFromInput());

    // Event Langkah Selanjutnya
```

```

        stepButton_3030.addActionListener(e -> performStep());

        // Event Reset
        resetButton_3030.addActionListener(e -> reset());

```

2.4.2 Pembacaan Input dan Validasi Data (setArrayFromInput)

- bertugas mengambil teks yang diketik pengguna, memotongnya berdasarkan tanda koma, dan mengubahnya menjadi deretan angka integer di memori.

```

private void setArrayFromInput() {
    String text = inputField_3030.getText().trim();
    if (text.isEmpty()) return;

    String[] parts = text.split(",");
    array_3030 = new int[parts.length];

    try {
        for (int k = 0; k < parts.length; k++) {
            array_3030[k] = Integer.parseInt(parts[k].trim());
        }
    } catch (NumberFormatException e) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang
dipisahkan "
            + "dengan koma!", "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    i_3030 = 1;
    sorting_3030 = true;
    stepButton_3030.setEnabled(true);
    stepArea_3030.setText("");

    panelArray_3030.removeAll();
    labelArray_3030 = new JLabel[array_3030.length];

    for (int k_3030 = 0; k_3030 < array_3030.length; k_3030++) {
        labelArray_3030[k_3030] = new
JLabel(String.valueOf(array_3030[k_3030]));
        labelArray_3030[k_3030].setFont(new Font("Arial", Font.BOLD, 24));

        labelArray_3030[k_3030].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_3030[k_3030].setPreferredSize(new Dimension(50, 50));

        labelArray_3030[k_3030].setHorizontalAlignment(SwingConstants.CENTER);

        panelArray_3030.add(labelArray_3030[k_3030]);
    }

    panelArray_3030.revalidate();
    panelArray_3030.repaint();
}

```

2.4.3 Otak Eksekusi Algoritma Per Langkah (performStep)

- program biasa yang langsung mengurutkan data sekaligus, kelompok ini memecah algoritma *Insertion Sort* agar berjalan **hanya satu langkah** setiap kali tombol diklik.

```

private void performStep() {
    if (i_3030 < array_3030.length && sorting_3030) {

```

```

        int key_3030 = array_3030[i_3030];
        j_3030 = i_3030 - 1;

        StringBuilder stepLog = new StringBuilder();
        Object stepCount;
        stepLog.append("Langkah ").append(stepCount_3030).
            append(": Memasukkan ").append(key_3030).append("\n");

        while (j_3030 >= 0 && array_3030[j_3030] > key_3030) {
            array_3030[j_3030 + 1] = array_3030[j_3030];
            j_3030--;
        }

        array_3030[j_3030 + 1] = key_3030;

        updateLabels();
        stepLog.append("Hasil:
    ").append(arrayToString(array_3030)).append("\n\n");
        stepArea_3030.append(stepLog.toString());

        i_3030++;
        stepCount_3030++;

        if (i_3030 == array_3030.length) {
            sorting_3030 = false;
            stepButton_3030.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
        }
    }
}

```

2.4.4 Sinkronisasi Komponen Visual (updateLabels & arrayToString)

- berfungsi sebagai jembatan penghubung antara data di memori dengan tampilan layar. Setiap kali algoritma performStep selesai merubah susunan angka di latar belakang, fungsi updateLabels akan langsung memperbarui teks di dalam kotak-kotak visual agar ikut berubah secara *real-time*.

```

private void updateLabels() {
    for (int k_3030 = 0; k_3030 < array_3030.length; k_3030++) {
        labelArray_3030[k_3030].setText(String.valueOf(array_3030[k_3030]));
    }
}

```

2.4.5 Pembersihan Sistem dan Gerbang Jalannya Program (reset & main)

- penutup ini memiliki dua tugas krusial. Prosedur reset bertanggung jawab untuk membersihkan seluruh komponen aplikasi, menghapus kotak angka di layar, mengosongkan riwayat teks log, serta mengembalikan penunjuk indeks ke posisi awal agar pengguna bisa memasukkan data baru.

```

private void reset() {
    inputField_3030.setText("");
    panelArray_3030.removeAll();
    panelArray_3030.revalidate();
    panelArray_3030.repaint();
    stepArea_3030.setText("");
    stepButton_3030.setEnabled(false);
    sorting_3030 = false;
    i_3030 = 1;
}

```

```
    stepCount_3030 = 1;  
}
```

BAB III

KESIMPULAN

3.1 Kesimpulan

Karakteristik dan Logika AlgoritmaPraktikum. Pekan ke-7 ini berhasil mengimplementasikan tiga algoritma pengurutan (*sorting*) dasar, yaitu *Bubble Sort*, *Insertion Sort*, dan *Selection Sort*. Ketiganya memiliki cara kerja yang unik: *Bubble Sort* mengapungkan angka terbesar ke ujung kanan melalui pertukaran tetangga secara berulang; *Insertion Sort* menyisipkan elemen ke posisi yang tepat seperti merapikan kartu di tangan; sedangkan *Selection Sort* memindai seluruh data untuk mencari nilai minimum lalu menukarnya ke posisi depan.

Efisiensi dan Performa Program. Meskipun ketiga algoritma memiliki kompleksitas waktu rata-rata sebesar $O(n^2)$ akibat penggunaan perulangan bertingkat (*nested loop*), performa aslinya di lapangan berbeda-beda. *Insertion Sort* terbukti paling adaptif dan efisien untuk data berskala kecil atau yang kondisinya sudah hampir terurut karena minimnya proses pergeseran. Sementara itu, *Selection Sort* unggul dalam konsistensi jumlah penukaran data yang sedikit, dan *Bubble Sort* menjadi yang paling lambat pada kasus data acak besar karena frekuensi pertukaran elemennya yang terlalu tinggi.

Manfaat Visualisasi Berbasis GUI. Penerapan algoritma *Insertion Sort* ke dalam aplikasi berbasis Graphical User Interface (GUI) menggunakan Java Swing memberikan manfaat edukasi yang besar. Dengan fitur eksekusi per langkah (*step-by-step*), pengguna dapat mengamati perubahan posisi angka, penentuan nilai *key*, dan pembaruan log teks secara langsung (*real-time*). Visualisasi interaktif ini mempermudah pemahaman alur logika jalannya program dibandingkan hanya melihat barisan kode atau output teks statis pada konsol biasa.

TUGAS PRATIKUM

“Mahasiswa_2511533030”

```
package pekan7_2511533030;

public class Mahasiswa_2511533030 {
    private String nama_3030;
    private String nim_3030;
    private String prodi_3030;

    // constructor
    public Mahasiswa_2511533030(String nama_3030, String nim_3030, String
prodi_3030) {
        this.nama_3030 = nama_3030;
        this.nim_3030 = nim_3030;
        this.prodi_3030 = prodi_3030;
    }

    // getter setter
    public String getNama() {
        return nama_3030;
    }

    public void setNama(String nama_3030) {
        this.nama_3030 = nama_3030;
    }

    public String getNim() {
        return nim_3030;
    }

    public void setNim(String nim_3030) {
        this.nim_3030 = nim_3030;
    }

    public String getProdi() {
        return prodi_3030;
    }

    public void setProdi(String prodi) {
        this.prodi_3030 = prodi;
    }

    @Override
    public String toString() {
        return nama_3030 + " - " + nim_3030 + " - " + prodi_3030;
    }
}
```

“SortingGUI_2511533030”

```
package pekan7_2511533030;

public class Mahasiswa_2511533030 {
    private String nama_3030;
    private String nim_3030;
    private String prodi_3030;
```

```

// constructor
public Mahasiswa_2511533030(String nama_3030, String nim_3030, String
prodi_3030) {
    this.nama_3030 = nama_3030;
    this.nim_3030 = nim_3030;
    this.prodi_3030 = prodi_3030;
}

// getter setter
public String getNama() {
    return nama_3030;
}

public void setNama(String nama_3030) {
    this.nama_3030 = nama_3030;
}

public String getNim() {
    return nim_3030;
}

public void setNim(String nim_3030) {
    this.nim_3030 = nim_3030;
}

public String getProdi() {
    return prodi_3030;
}

public void setProdi(String prodi) {
    this.prodi_3030 = prodi;
}

@Override
public String toString() {
    return nama_3030 + " - " + nim_3030 + " - " + prodi_3030;
}
}

```

Output

Pengurutan Nama Mahasiswa - Pekan 7

Form Input Mahasiswa

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Pilih Algoritma Sorting: Insertion Sort

NIM	Nama Mahasiswa	Program Studi
2511533020	Budi	Informatika
2511533010	Maya	Informatika
2511533030	Risky	Informatika

Visualisasi Proses Langkah demi Langkah

— INSERTION SORT —

Awal : [Risky - 25115330 - Informatika, Maya - 2511533010 - Informatika, Budi - 2511533020 - Informatika]

Langkah 1 : [Maya - 2511533010 - Informatika, Risky - 25115330 - Informatika, Budi - 2511533020 - Informatika]

Langkah 2 : [Budi - 2511533020 - Informatika, Maya - 2511533010 - Informatika, Risky - 25115330 - Informatika]

Status: Pengurutan Sukses Berhasil!!

Pengurutan Nama Mahasiswa - Pekan 7

Form Input Mahasiswa

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Tambah Data

Kosongkan Semua Data

Pilih Algoritma Sorting:

Selection Sort

Mulai Sorting

NIM	Nama Mahasiswa	Program Studi
2511533020	Budi	Informatika
2511533010	Maya	Informatika
25115330	Rizky	Informatika

Visualisasi Proses Langkah demi Langkah

```
=== SELECTION SORT ===
Awal : [Budi - 2511533020 - Informatika, Maya - 2511533010 - Informatika, Rizky - 25115330 - Informatika]

Pass 1  : [Budi - 2511533020 - Informatika, Maya - 2511533010 - Informatika, Rizky - 25115330 - Informatika]
Pass 2  : [Budi - 2511533020 - Informatika, Maya - 2511533010 - Informatika, Rizky - 25115330 - Informatika]

Status: Pengurutan Sukses Berhasil!!
```



Pengurutan Nama Mahasiswa - Pekan 7

Form Input Mahasiswa

NIM Mahasiswa:

Nama Mahasiswa:

Program Studi:

Tambah Data

Kosongkan Semua Data

Pilih Algoritma Sorting:

Bubble Sort

Mulai Sorting

NIM	Nama Mahasiswa	Program Studi
2511533020	Budi	Informatika
2511533010	Maya	Informatika
25115330	Rizky	Informatika

Visualisasi Proses Langkah demi Langkah

```
=== BUBBLE SORT ===
Awal : [Budi - 2511533020 - Informatika, Maya - 2511533010 - Informatika, Rizky - 25115330 - Informatika]

Pass 1  : [Budi - 2511533020 - Informatika, Maya - 2511533010 - Informatika, Rizky - 25115330 - Informatika]
Pass 2  : [Budi - 2511533020 - Informatika, Maya - 2511533010 - Informatika, Rizky - 25115330 - Informatika]

Status: Pengurutan Sukses Berhasil!!
```

